

Empirical Integration of a Formal Network Verifier as an Oracle inside an LLM

Generate–Verify–Repair Loop: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired within-task experiment (study_id netverify_gvr_empirical_integration_2026_v1) that embeds a deterministic network verifier as an oracle inside a bounded LLM Generate–Verify–Repair (GVR) pipeline. Using a synthetic intent→configuration suite (N=270 pairs), gpt-5-mini as the generator, and a Dockerized Kathará/Mininet emulator as ground truth, the run used shared_initial_candidate semantics (one generated candidate per pair evaluated in both arms). Observed outcomes: baseline accepted 270/270 (100%); guarded (verify → optional bounded repair) accepted 270/270 (100%); no verifier rejections occurred and no repairs were invoked. Paired difference = 0.00 (paired bootstrap 95% CI [0.00,0.00]; exact McNemar two-sided p = 1.0; bootstrap resamples = 10,000, seed = 1729). We document the preregistration ↔ executed-design mismatch (power calculations assumed independent per-arm generation) and provide artifact-anchored instructions, hashes, and a minimal recipe to repro and to run the independent-sampling experiment the preregistration originally targeted. All raw outputs, validator traces, and per-artifact SHA-256 manifests are archived for deterministic reproduction.

I. INTRODUCTION

Motivation. Natural-language intent→device configuration automation offers large productivity gains but creates safety risk: an LLM may emit syntactically plausible commands that break invariants (reachability, redundancy, service continuity). A common operational mitigation is the Generate–Verify–Repair (GVR) architecture: probabilistic generation, deterministic verification, and bounded repair guided by verifier diagnostics. Scientific gap. Prior work motivates GVR conceptually and reports case studies and toolchains, but there is little preregistered, artifact-anchored evidence quantifying the marginal effect of inserting a deterministic verifier into an LLM pipeline under fixed budgets and sampling semantics. Goals and contribution. We preregistered a confirmatory paired experiment (study_id netverify_gvr_empirical_integration_2026_v1), archived every artifact, executed the frozen plan, and release the full provenance. The paper reports the executed conditional outcome (shared_initial_candidate semantics) and documents the precise minimal changes required to test the original independent-sampling estimand.

II. RELATED WORK

Generate→verify architectures appear across regulated code generation and LLM-agent literature [Cadenas et al., Convergent Correctness; doi:10.2139/ssrn.6754899] and formal network verification (NetKAT/weighted NetKAT decision procedures) provides canonical deterministic oracles for many network properties [Suárez Acevedo et al.; doi:10.1145/3808318]. Surveys on LLMs/agents for NetOps and RAG contextualization (OpenAlex entries W4414359351, W4400072049, W4386722061) informed our task design and verification choices. Our work is experimental: a preregistered, fully archived paired test of verifier insertion with deterministic logging and a reproducible minimal follow-up recipe.

III. METHODOLOGY

Preregistration and frozen plan. The preregistration (study_id netverify_gvr_empirical_integration_2026_v1; preregistration JSON SHA-256 8ae7fbadf89e5d56671560e43f03c4a92ad5f8c9b8586ac8eb3cc3c8fa00a1c0) fixed estimands, sampling plan (Npairs=270), generator and verifier identifiers, budgets, retry policies, and stopping rules before execution. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (within-task difference in verifier detection/accept rates). Design choices executed (frozen): generator declared gpt-5-mini (execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82) called via recorded provider openai_responses; deterministic verifier deterministic_netops_validator_v5 executed against Dockerized Kathará/Mininet emulator (emulator images and test harness recorded under execution/*). Crucial executed sampling semantics (disclosed and logged): execution_contract.independent_condition_generation = false (shared_initial_candidate semantics): a single generated candidate per task was evaluated in both arms. This is the decisive design choice that makes the executed estimand conditional on identical generated candidates; the original preregistered power/MDE calculations assumed independent per-arm generation. Task bank. SyntheticNetOpsIntentConfig_v1: 270 intent→configuration tasks (3 vendor-style syntaxes ×

3 topology templates $\times$ 30 tasks). Each task includes initial_state, intent, required_sequence, and validator constraints; representative examples and all JSONs are archived under execution/* (see representative scoring JSONs under execution/scoring/). Execution and logging. All model calls, provider traces, raw responses, validator traces, scoring JSONs, and SHA-256 hashes were deterministically recorded (execution/execution_manifest.json; execution/raw_results.jsonl SHA-256 43381c4b07e1ffabb163657f56ae893fd65fc12d5307ea1ff3bb98f7cf43329d). Analysis plan. Primary test: paired binary comparison (exact McNemar two-sided) and paired bootstrap percentile 95% CI with 10,000 resamples (seed=1729). Failure and retry policy: up to two retries per failed provider call; no infrastructure restarts were needed and failed_hosted_model_call_event_count=0.

IV. RESULTS

Execution summary (artifact-anchored). Planned episodes: 540 (270 pairs). Completed: 540; failed: 0. Model calls used: 270 (one shared generation per pair, because independent_condition_generation=false). Scoring aggregate (analysis/condition_summary.csv SHA-256 9f43d7b4...): baseline accepts = 270/270 (100%); guarded accepts = 270/270 (100%). Paired contingency (analysis/paired_contingency_table.csv SHA-256 62cd99d7...): n11=270, n10=0, n01=0, n00=0. Statistical inference: paired difference = 0.00; exact McNemar two-sided $p = 1.0$; paired bootstrap 95% percentile CI = [0.00,0.00] (10,000 resamples, seed=1729). Repair activity: no verifier rejections—bounded repair path not invoked. Verifier $\leftrightarrow$ emulator concordance (validation subset 54 tasks): concordance 54/54 (false_accept=0; false_reject=0) in archived scoring JSONs. Representative example artifacts: execution/scoring/task-000001-guarded.json (SHA-256 350bfaa2...) contains full validator_trace for a representative episode; execution/raw_results.jsonl (SHA-256 43381c4b...) contains all raw model transcripts.

V. DISCUSSION

Interpretation and preregistration/design mismatch. The observed degenerate contingency (zero discordant pairs) arises because a single candidate per task was generated and that candidate satisfied the verifier rule set for every task; hence the verifier could not change accept/reject counts under the executed shared candidate semantics. The preregistered power and MDE calculations assumed independent per-arm generation; because execution_contract.independent_condition_generation=false, those power calculations are not applicable to the executed run. This manuscript therefore reports a conditional null result: when the chosen generator/configuration produces verifier-acceptable candidates for a given task bank, inserting the verifier (with the chosen rule set and a one-call repair budget) will not change observed accept/reject counts under shared candidate evaluation. Artifact-anchored verifier rule

set pointer. The deterministic verifier configuration and rule set used by deterministic_netops_validator_v5 are archived; see execution/transformation_manifest.json (SHA-256 ecd9236e84c744f3f0696d8fe96cbfe73968822be9aa153bf1e313f4fccc731743329d) for the exact verifier rule file(s) and the invocation commands used by the harness. Minimal reproducible recipe to test the original independent-sampling estimand (artifact-anchored). To test the preregistered estimand with independent per-arm sampling, make only these deterministic edits in the execution_manifest and re-run the harness: (1) set execution_contract.independent_condition_generation = true; (2) set execution_contract.maximum_model_calls $\geq$ 540 to allow one independent generation per arm per task; (3) set a nonzero temperature in execution/model_configuration.json or provide per-call seeds to ensure sampling variability; (4) optionally increase maximum_repair_calls_per_task > 1 to evaluate bounded-repair convergence. All file paths and field names above exist in the released artifacts (execution/execution_manifest.json and execution/model_configuration.json). Operational implication. The run provides a concrete boundary condition: under the chosen synthetic task bank and generator settings, verifier insertion produced no marginal effect. This clarifies that empirical evaluations of GVR must make sampling semantics explicit and align power calculations accordingly.

VI. LIMITATIONS

- Synthetic task suite and emulator fidelity: tasks are synthetic and executed in a Dockerized Kathará/Mininet environment; production devices and multi-vendor heterogeneity may produce different verification/repair outcomes.
- Single model and single verifier rule set: only gpt-5-mini and deterministic_netops_validator_v5 were used; results are not multi-model or multi-verifier generalizations.
- Sampling semantics vs preregistered power: the preregistered power/MDE assumed independent per-arm sampling; the executed run used shared_initial_candidate semantics, so the original power calculations are inapplicable to the observed conditional contingency.
- Bounded repair budget: the repair loop allowed at most one repair call per rejected candidate; multi-attempt repair strategies may yield different repair success rates.
- Emulator $\leftrightarrow$ production gap: verifier acceptance against an emulator does not guarantee identical behavior or safety on production hardware and in operational processes.

VII. CONCLUSION

We preregistered and executed a fully archived paired study that integrated a deterministic network verifier into an LLM GVR pipeline and published the complete provenance. Under the executed shared_initial_candidate semantics (one generated candidate per pair), the generated candidates satisfied the verifier rule set for all $N=270$ tasks so the verifier/repair path was not invoked and the paired difference is 0.00. The key methodological lesson is that generation semantics (shared

candidate vs independent sampling) fundamentally determine whether a verifier insertion is testable under a preregistered paired estimand and whether the preregistered MDE applies. We release the full artifact bundle and a minimal, deterministic recipe (artifact pointers and required manifest edits) so others can rerun the archived harness either under shared candidate semantics (as executed) or under independent per-arm sampling to test the original estimand. All artifacts, logs, and per-artifact SHA-256 hashes are archived for deterministic reproduction (see `execution/*` and `analysis/*`).

DISCLOSURE STATEMENT

Mandatory Disclosure Statement (artifact-backed). Initial master prompt reference (immutable archive): `provenance/master_prompt.txt` SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd51582b391b51a. Preregistration: `study_id` `netver-ify_gvr_empirical_integration_2026_v1` (preregistration JSON) SHA-256 8ae7fbadf89e5d56671560e43f03c4a92ad5f8c9b8586ac8eb3cc3c86ad0216084c. Declared LLM and configuration: `gpt-5-mini` (execution/model_configuration.json) SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82. Called via provider recorded as `openai_responses`. Orchestration framework and adapter: `adapter_family` hosted `netops_gvr_v1`; execution manifest and full call traces archived under `execution/*` (execution/raw_results.jsonl) SHA-256 43381c4b07e1ffabb163657f56ae893fd65fc12d5307ea1ff3bb98f7cf43329a. Verifier rule set and configuration pointer (artifact-anchored): the deterministic verifier configuration and rule file(s) used by `deterministic_netops_validator_v5` are recorded in `execution/transformation_manifest.json` (SHA-256 ecd9236e84c744f3f0696d8fe96cbfe73968822be9aa153bf1e313f4fcc717) and the per-episode scoring JSONs under `execution/scoring/` contain `validator_trace` entries demonstrating how the verifier was applied (examples: `execution/scoring/task-000001-guarded.json` SHA-256 350bfaa2eec0deb0e16121f5e7effa2d8743d28f3f014856107acfabb42487e). Topic constraints and human role: prior to the autonomy lock the Research Director constrained the broad topic family to Generative AI and LLMs for NetOps; after the autonomy lock the autonomous pipeline performed literature review, study selection, preregistration, code generation, execution, analysis, and manuscript drafting without human manual editing of technical content. Preregistered stopping, retry, and failure handling: frozen and archived in the preregistration and execution manifests; model/API call retries allowed up to two attempts; no retries were necessary in this run and `failed_hosted_model_call_event_count=0`. Sampling semantics decisive to outcome: the executed run used `execution_contract.independent_condition_generation=false` (`shared_initial_candidate` semantics) so a single generator candidate per task was evaluated in both arms; this design choice is logged and is the proximate reason the verifier path was not invoked for any pair. Reproducible minimal recipe to

test the original preregistered independent-sampling estimand:

- (1) set `execution_contract.independent_condition_generation = true` in the archived execution manifest (`execution/execution_manifest.json`),
 - (2) set `execution_contract.maximum_model_calls >= 540`,
 - (3) set a nonzero temperature or provide per-call seeds in `execution/model_configuration.json` to induce sampling variability,
 - (4) optionally increase `maximum_repair_calls_per_task > 1`.
- All field names and paths referenced above exist in the released artifact bundle. Deterministic artifacts and hashes: all raw responses, provider traces, scoring JSONs, validator traces, execution logs, and per-artifact SHA-256 hashes are archived under `execution/*` and `analysis/*`; representative hashes: `execution/raw_results.jsonl` SHA-256 43381c4b07e1ffabb163657f56ae893fd65fc12d5307ea1ff3bb98f7cf43329a; `execution/model_configuration.json` SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82; `execution/transformation_manifest.json` (verifier rule set pointer) SHA-256 ecd9236e84c744f3f0696d8fe96cbfe73968822be9aa153bf1e313f4fcc717; `analysis/paired_contingency_table.csv` SHA-256 62cd99d7bf35cab7ee751f4faae9004784597d06684b884750e66083a3e52f2. Full details of peer review and revision: the autonomous pipeline executed internal critique and revision steps; no human manually edited the technical manuscript content after the autonomy lock. Execution timestamps: `started_at_utc` 2026-08-26T22:36:00.638883+00:00; `completed_at_utc` 2026-08-26T23:07:33.563594+00:00. Limitations and scope are declared in the manuscript (section Limitations).

REFERENCES

- [1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.1179/ssrn.6754899
- [2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318
- [3] Mohammad Raza, Nataša Milić-Frayling, "Instantiation-based Formalization of Logical Reasoning Tasks Using Language Models and Logical Solvers", 2025, 10.24963/ijcai.2025/516
- [4] Abdelkader Mekrache, Adlen Ksentini, Christos Verikoukis, "Intent-Based Management of Next-Generation Networks: an LLM-Centric Approach", 2024, 10.1109/mnet.2024.3420120
- [5] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Dejan Kostić, Marco Chiesa, "Making Network Configuration Human Friendly", 2023, 10.48550/arxiv.2309.06342